

1. Codebase Maintainability & Scalability

What strategies would you implement to ensure the codebase remains scalable, maintainable, and easy to navigate as the project grows?

a) Modular Folder Structure

- I used a modular folder structure to accommodate for easy maintainability and collaboration.

b) Separation of Concerns

- The original code mixes UI rendering, Network requests and State management.
- In contrast, my optimized code separates:
 - **API calls** - hooks/
 - **UI logic** - components/
 - **Global state** - store/
- This makes it easier to test, debug, and extend.

c) Custom Hooks for Data Fetching

- I added custom hook *useDashboardData* to:
 - Centralize fetching projects and users.
 - To enable easy reuse across different pages.
 - To keep components clean and focused on user interface.

d) TypeScript

- Unlike the original code, my code uses TypeScript, which:
 - Helps to avoid runtime bugs during development.
 - Improves autocompletion to accelerate code development.
 - Documents data shapes for new developers.

e) Pagination

- My code also handles client side pagination.

- Server side pagination was not possible with the free API used, I could not track total items. This would have been sufficient to avoid overwhelming the client in the case of users and project scale.

f) Component Reusability

- I used a generic reusable component `<Table />` that accepts any data type.
 - This makes it reusable across different tables in the app — big win for scalability.
-

Have you worked on a large-scale React project where the codebase became difficult to manage? What challenges did you face, and what steps did you take to improve maintainability?

Yes. Some of the most common challenges I've faced are listed below:

Problem: Components becoming bloated with logic

Solution: Split into smaller components, introduced custom hooks

Problem: Difficult state sharing

Solution: Adopted Redux for centralized but modular state

Problem: API logic scattered in components

Solution: Moved logic to a central location and built typed wrappers with Axios.

2. Enforcing Code Consistency

How do you ensure that multiple developers follow consistent coding patterns and best practices when contributing to the same React codebase?

a) Code Reviews

- Enforce a strict code review process via PRs.
- Use GitHub/GitLab merge rules, where at least 1 review is required.

b) Introduce well-defined Coding Guidelines

- Maintain a living README in the repo that defines:
 - Folder structures
 - Naming conventions

c) Shared Component Libraries

- Use a UI library to standardize components' look and feel.

What tools, methodologies, or workflows would you recommend to maintain code consistency and scalability?

- Testing (Jest + React Testing Library) - To catch regressions early
- Redux Toolkit - To manage complex shared state cleanly
- TypeScript - To prevent runtime errors and enforce interfaces
- Prettier + ESLint - To auto-format and enforce lint rules